C++14 Fundamentals - Student Edition

Week 4 - Class 8: Bitwise Operations and Overflow (Aligned Tables Edition)

Why This Matters

Bits are the foundation of data. With bitwise operators and shifting you can:
- Control individual flags.
- Pack/unpack multiple small values.
- Perform fast arithmetic by powers of 2.
- Implement low-level protocols.

Overflow rules are equally important to avoid Undefined Behavior and security bugs.

Masks: Definition and Uses

A mask is a binary pattern that highlights specific bits. Use with bitwise operators:
- AND (&): isolate bits (others cleared)
- OR (|): set bits (force to 1)
- XOR (^): toggle bits (flip)
- NOT (~): invert bits (often used to clear)

Mask Examples

AND (isolate): x=10101010, mask=00001111 -> result=00001010.
OR (set) : x=10100000, mask=00001111 -> result=10101111.
XOR (toggle) : x=10101010, mask=00001111 -> result=10100101.
NOT (clear) : x=10101111, mask=00001111, ~mask=11110000 -> x & ~mask = 10100000.

Mask Summary Table

Operation	Code Example	Effect
AND &	res = x & mask	Isolate bits where mask = 1
OR |	res = x | mask	Set bits where mask = 1
XOR ^	res = x ^ mask	Toggle bits where mask = 1
NOT ~	res = x & ~mask	Clear bits where mask = 1

Bitwise Operators: Truth Table (per bit)

a	b	~a	~b	a&b	a|b	a^b
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	1	0

Example Code: Bitwise Operators

```
#include <iostream>
#include <bitset>
using namespace std;
int main(){
    unsigned a = 0b1100; // 12
    unsigned b = 0b1010; // 10
    cout << "a & b = " << bitset<4>(a & b) << "\n"; // 1000
    cout << "a | b = " << bitset<4>(a | b) << "\n"; // 1110
    cout << "a ^ b = " << bitset<4>(a ^ b) << "\n"; // 0110
```

```
    cout << "~a    = " << bitset<4>(~a & 0b1111) << "\n"; // 0011
}
```

Bit Shifting: What, Why, When

Left shift (<<): moves bits left, fills right with zeros, multiply by 2^k.
Right shift (>>): moves bits right. Unsigned: fills left with zeros (divide by 2^k). Signed negative: implementation-defined.

Why shift?
- Multiply/divide by powers of 2 quickly.
- Align or position values in bit fields.
- Extract parts of numbers.
- Create masks.

ASCII Shift Examples

Value: 00110100 (52)
<<2: 11010000 (208)
>>2: 00001101 (13)

Example Code: Shifting

```
#include <iostream>
#include <bitset>
using namespace std;
int main(){
    unsigned x = 0b00110100; // 52
    cout << "x      = " << bitset<8>(x) << "\n";
    cout << "x << 2 = " << bitset<8>(x << 2) << "\n";
    cout << "x >> 2 = " << bitset<8>(x >> 2) << "\n";
}
```

Shift Safety

- Never shift by >= width (Undefined Behavior).
- Left shift negative values is Undefined Behavior.
- Right shift negatives is implementation-defined.
- Prefer unsigned fixed-width (uint32_t, uint64_t).

Bit Fields: Packing and Unpacking

A bit field stores multiple small values inside one integer.
Example: RGB packed in one byte (3+3+2 bits).

```
#include <iostream>
#include <bitset>
#include <cstdint>
using namespace std;
int main(){
    uint8_t r=5,g=2,b=3; // 0..7,0..7,0..3
    uint8_t pixel=(r<<5)|(g<<2)|b;
    cout<<bitset<8>(pixel)<<"\n";
    uint8_t ur=(pixel>>5)&0b111;
    uint8_t ug=(pixel>>2)&0b111;
    uint8_t ub=pixel&0b11;
    cout<<(int)ur<<" "<<(int)ug<<" "<<(int)ub<<"\n";
}
```

Overflow: Signed vs Unsigned

Unsigned overflow: wraps modulo 2^N.

Signed overflow: Undefined Behavior (unpredictable).

Bit Tricks Without If Statements

Using bitwise math instead of if conditions.
Example: test if power of two.
bool isPowerOfTwo(unsigned x){ return x!=0 && (x&(x-1))==0; }

Common Mistakes

1. Misunderstanding AND vs OR for masks.
2. Printing uint8_t as char.
3. Shifting by >= width (Undefined Behavior).
4. Left shift of negative (Undefined Behavior).
5. Relying on right shift of negatives (not portable).
6. Forgetting to use unsigned types.

Exercises

1. Practice AND/OR/XOR/NOT with small values.
2. Demonstrate shift multiplication/division.
3. Pack/unpack RGB values.
4. Show unsigned wrap vs signed overflow.
5. Implement power-of-two check.
6. Build flags READ/WRITE/EXEC system.

Project Tie-In

Extend Type Explorer with a Bit Lab:
- Display bit patterns.
- Show mask operations.
- Pack/unpack demo.
- Show overflow behavior.